Software Engineering Lab 5

Static Code Analysis of the Pharmacy Management System

Sai Sathvik Gullipalli | PES2UG24CS430 | Section G

Individual submission | 21 October 2026

The assigned pharmacy_manager.py was analyzed with Pylint, Bandit and Flake8. All reported findings were addressed. The final application passes all three tools and 27 regression tests. The following tables document 19 groups of findings, including additional behavior defects found during review.

Verification Results

Check	Original	Final
Pylint	4.83/10; 45 messages	10.00/10; 0 messages
Bandit	8: 2 high, 2 medium, 4 low	0 findings
Flake8	22 violations	0 violations
Regression tests	Not supplied upstream	27 passed

All final commands returned exit code 0. No warning rules were disabled. Python 3.11.9; Pylint 4.1.3; Bandit 1.9.4; Flake8 7.4.1. An empty Flake8 report indicates a successful run.

Issue Table

Line numbers refer to the unchanged original script in reports/before/pharmacy_manager.py. Repeated diagnostics are grouped by cause. Severity labels in the security table are Bandit's reported ratings.

Security Findings

Issue	Type	Lines	Description	Fix approach
Shell command injection	High: Bandit B605	68	os.system concatenated a medicine name into a shell command.	print_label prints the text directly; shell syntax remains literal.
Weak MD5 hash	High: Bandit B324	50-51	The patient identifier used MD5 and ambiguous name + phone concatenation.	Hash a JSON array of the fields with SHA-256. This remains an identifier, not anonymization.
Unrestricted eval	Medium: Bandit B307; Pylint W0123	46-48	A formula could execute arbitrary Python code through eval.	Parse an AST; allow only numbers, signs and +, -, *, /. Bound input size, tree size and results.
Unsafe pickle loading	Medium: B301; Low: B403	3, 56-65	pickle.load can execute code from an untrusted stock file.	Use JSON and validate every record before replacing stock. Old pickle files are not loaded.
Embedded credential	Low: Bandit B105	11, 78-82, 102	The same source-code password was used for every installation.	Read PHARMACY_PASSWORD from the environment, deny login when unset and use compare_digest.
Predictable prescription reference	Low: Bandit B311	53-54	random.randint generated a predictable six-digit prescription reference.	Use secrets.token_hex(16) for a 128-bit reference. Randomness alone does not guarantee uniqueness.
Silently swallowed errors	Low: B110; W0702; E722	22-28	A bare except followed by pass hid missing medicines and other failures.	Raise KeyError or ValueError as appropriate. The demo catches specific failures and logs them.

Code Quality and Style

Issue	Type	Lines	Description	Fix approach
Shared default audit list	Pylint W0102	13-20	audit=[] was shared by calls that omitted the argument.	Use audit=None and create a fresh list per call; preserve an explicitly supplied audit list.
Manual file cleanup	Pylint R1732	56-65	Manual open/close pairs could leave files open after an exception.	Use with blocks and explicit UTF-8 encoding for JSON files.
Global dictionary rebinding	Pylint W0603	61-65	loadStock rebound the global medicines dictionary to unchecked input.	Validate into a temporary dictionary, then clear and update the existing dictionary only on success.
Inconsistent return behavior	Pylint R1710	13-20	addMedicine returned None for an empty name and a list on success.	Invalid input raises ValueError. Successful add_medicine calls always return the audit list.
Redundant boolean branches	Pylint R1703, R1705	33-38, 78-82	if/else branches returned True and False directly.	Return the expiry comparison and the credential comparison directly after validation.
Missing documentation and naming	Pylint C0114, C0116, C0103	1, 13-89	The module/functions lacked docstrings and function names used camelCase.	Add concise docstrings and rename functions and their callers to snake_case.
Unused imports	Pylint W0611; Flake8 F401	2, 6	sys and logging were imported without being used.	Remove sys. Use logging for rejected demo actions and configure it only in the main block.
Formatting and long lines	C0301, C0209; E302, E305, E501	13-108	Definitions lacked blank lines; output lines were too long; an audit string used old formatting.	Use f-strings, two blank lines around top-level definitions and lines no longer than 79 characters.

Additional Behavior Findings

Issue	Type	Lines	Description	Fix approach
Invalid stock and dispensing	Manual review and regression tests	13-24, 90-95	The demo accepted negative quantities and a numeric name. Excess or negative dispensing could corrupt stock.	Validate names, integer quantities, finite prices and dates; reject unknown, expired or insufficient stock before mutation.
Mixed stock batches	Manual review and regression tests	16-18	Restocking silently kept the first price and expiry even when the new batch differed.	Reject a changed price or expiry for an existing name; require a distinct name for a different batch.
Expiry and billing boundaries	Manual review and regression tests	40-44, 70-75	expiringSoon included already expired stock and excluded the last day. Billing ignored expiry and repeated-item availability.	Use 0 <= days remaining <= window. Validate bill items, combine repeated quantities and block expired or unavailable stock.
Execution during import	Manual review and regression tests	108	An unconditional main() printed output, changed stock and wrote a file whenever the module was imported.	Use the __name__ == '__main__' guard. Import now has no demo or file-writing side effects.

Validation

The 27 passing tests cover stock validation, dispensing, expiry boundaries, billing, audit isolation, safe arithmetic, literal labels, JSON round trips, failed-load atomicity, credentials and import safety. Detailed results are in test_results.txt.

Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest changes were removing the unused sys import, adding blank lines and docstrings, and replacing audit=[] with audit=None. Their causes and fixes were local and easy to verify. The hardest change was removing eval while preserving arithmetic such as 250 * 2. I used an AST allowlist for numeric literals, arithmetic operators and unary signs, with size and result limits. I tested valid formulas as well as calls, attributes, invalid syntax and division by zero. Replacing pickle also required care: the entire JSON file must pass validation before it can replace the current stock.

2. Did the static analysis tools report any false positives? If so, describe one example.

I did not establish a definite false positive that justified suppressing a warning. Bandit's B311 warning on random.randint is context-dependent: the prescription number was a reference, not an authentication token, so the security severity depends on how that reference is used. I still replaced it with secrets.token_hex and a larger identifier space. Similarly, the MD5 patient identifier was not a password hash; I replaced it with SHA-256 and unambiguous field encoding without claiming that hashing anonymizes patient information. Every reported warning was addressed, with no suppression comments.

3. How would you integrate static analysis tools into your actual software development workflow?

I would install pinned versions in a project virtual environment and run Flake8, Pylint and Bandit before committing. In continuous integration, I would run the same commands on each pull request and require the regression tests to pass. I would review high and medium severity security findings first, then correctness and style warnings. Any proposed suppression would need a documented reason and review. I would keep the tool output as evidence and use tests alongside analysis, because a clean static report does not prove that stock handling or expiry rules are correct.

4. What tangible improvements did you observe in code quality, readability, or potential robustness after applying the fixes?

Pylint improved from 4.83/10 with 45 messages to 10.00/10 with no messages. Bandit dropped from eight findings, including two high and two medium severity findings, to zero. Flake8 dropped from 22 violations to zero. The corrected demo rejects negative stock, numeric medicine names and unknown dispensing requests. The 27 passing regression tests cover audit isolation, expiry boundaries, billing, safe arithmetic, literal label printing, JSON loading, login behavior and import safety. Function names and docstrings now make the operations easier to follow, and specific exceptions make failures visible.

Reproduction and Sources

Required deliverables are pharmacy_manager.py, issues.md and reflection.md. The package also includes original and final tool reports, tests, pinned tool versions and README commands. verification.json records the source hash and exit codes.

Assigned repository: https://github.com/SETAPESU26/L5_55_pharmacy-manager

Original commit: ff16b8303037de5c50851ccb75916cf5870026b7

Assignment mapping: 5G_Lab-5-Repo-Assignemnt.xlsx, Sheet1 row 55, B55:D55. Requirements: Lab_5_Static_Code_Analysis_Student_handout.docx. Reflection questions: reflection_lab5.txt.